More practice - Graph traversal: BFS and DFS

3 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3
MEDIUM

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

More practice - Graph traversal: BFS and DFS | 3 questions | tap a title to jump to it

M 1. Compute distance to nearest taxi in grid — **PracHub 2026-01-10**

M 2. Solve matrix groups and recipe inventory — **PracHub 2026-01-22**

M 3. Validate parent array forms a tree — **PracHub 2026-02-11**

STUDY PATH: question 91 of 167 (more practice)

Priority score 2.46 | Graph traversal: BFS and DFS | asked 283 days ago

MEDIUM | IN APP LC 542 | PracHub | Technical Screen | 1 of 3

Compute distance to nearest taxi in grid

POSTED 2026-01-10 | 283 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 36.5 | 30 DAYS 0 | PRIORITY 102/228

ABOUT THIS QUESTION

Topic: More practice - Graph traversal: BFS and DFS

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #542 01 Matrix

LeetCode match: LC 542 01 Matrix (inferred from overview)

Posted on PracHub: 2026-01-10 (updated 2026-06-20)

Original: https://prachub.com/coding-questions/compute-distance-to-nearest-taxi-in-grid

QUESTION

Overview: This question evaluates understanding of graph traversal and shortest-path reasoning on grids, algorithmic problem-solving for computing distances from many targets, and the ability to manage time and space complexity on large inputs.

APPROACH

Multi-source BFS from all taxis simultaneously; the BFS level is the distance for each cell. O(mn).

HOW TO THINK ABOUT IT

Asking for the distance from every cell to its nearest source is one BFS started from all sources at once, not one BFS per cell. Put all taxis in the queue at distance 0; the first time the search reaches a cell is its shortest distance, because BFS visits cells in order of distance.

Basis: LeetCode 542, 01 Matrix, with 'T' for a taxi, '#' for a blocked cell and '.' for open ground. Distance counts moves through open cells (4-directional).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def nearest_taxi(grid, start):
    queue, seen = deque([(start, 0)]), {start}
    while queue:
        cell, d = queue.popleft()
        if is_taxi(grid, cell):
            return d
        for nxt in neighbours(grid, cell):
            if nxt not in seen:
                seen.add(nxt)
                queue.append((nxt, d + 1))
    return -1
answer = {}
for c in cells:
    answer[c] = nearest_taxi(grid, c)
```

BRUTE FORCE

TIME	SPACE
O((m * n)^2)	O(m * n)

WHY IT IS SLOW

A separate BFS per cell, O((m * n)^2). One BFS that starts from all taxis at once reaches every cell at its nearest-taxi distance, O(m * n).

OPTIMAL SOLUTION · One multi-source BFS

TIME	SPACE
O(m * n)	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A new taxi appears.

Start a BFS from the new taxi but only continue into cells where you improve the stored distance. The distances can only decrease, so the work is bounded by the cells whose answer actually changes.

FOLLOW-UP 2 Taxis move or disappear.

Distances can increase, which the decrease-only update cannot handle. Rebuild with one multi-source BFS (O(mn)), perhaps lazily every few updates and accepting bounded staleness between rebuilds.

FOLLOW-UP 3 Return which taxi is nearest, not only the distance.

Store a source id with each cell in the BFS: a cell copies the id of the cell that first reached it. Ties go to whichever reached it first, so say how you want ties broken.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute distance to nearest taxi in grid
#
# This question evaluates understanding of graph traversal
# and shortest-path reasoning on grids, algorithmic
# problem-solving for computing distances from many
# targets, and the ability to manage time and space
# complexity on large inputs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A new taxi appears.
# 2. Taxis move or disappear.
# 3. Return which taxi is nearest, not only the distance.
#
# INTERVIEW NOTES
```

```python
# Asking for the distance from every cell to its nearest
# source is one BFS started from all sources at once, not
# one BFS per cell. Put all taxis in the queue at distance
# 0; the first time the search reaches a cell is its
# shortest distance, because BFS visits cells in order of
# distance.
#
# BRUTE FORCE (say it first): BFS from every cell to find
# its nearest taxi
# For each open cell run a BFS until the first taxi is
# met. Correct, but it repeats almost the same search from
# every cell. Time O((m * n)^2). Space O(m * n).


from collections import deque


# BRUTE FORCE (say it first): BFS from every cell to find
# its nearest taxi | time O((m * n)^2)
def taxi_distances_brute(grid):
    R, C = len(grid), len(grid[0])
    out = [[-1] * C for _ in range(R)]
    for r in range(R):
        for c in range(C):
            if grid[r][c] == "#":
                continue
            seen, queue = {(r, c)}, deque([(r, c, 0)])
            while queue:
                x, y, d = queue.popleft()
                if grid[x][y] == "T":
                    out[r][c] = d
```

```python
                    break
                for nx, ny in (
                    (x + 1, y),
                    (x - 1, y),
                    (x, y + 1),
                    (x, y - 1),
                ):
                    if (
                        0 <= nx < R
                        and 0 <= ny < C
                        and grid[nx][ny] != "#"
                        and (nx, ny) not in seen
                    ):
                        seen.add((nx, ny))
                        queue.append((nx, ny, d + 1))
    return out


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def spread(grid, dist, queue, better=False):
    # grow outward; with better=True a cell may be updated
    # when a
    # new, closer source reaches it (distances only go
    # down)
    R, C = len(grid), len(grid[0])
    while queue:
        r, c = queue.popleft()
        for dr, dc in STEPS:
            a, b = r + dr, c + dc
            if not (0 <= a < R and 0 <= b < C):
                continue
            if grid[a][b] == "#":
                continue
            new = dist[r][c] + 1
            unseen = dist[a][b] == -1
            if unseen or (better and new < dist[a][b]):
                dist[a][b] = new
                queue.append((a, b))
```

```python
# BASE QUESTION (optimal): One multi-source BFS
# Idea: Initialise the queue with every taxi at distance 0
```

```python
# and expand in rounds. A cell receives its distance the
# first time it is reached; blocked cells are never
# entered; cells that no taxi can reach stay at -1.
def taxi_distances(grid):
    # 'T' taxi, '.' road, '#' wall. One BFS that starts
    # from
    # every taxi at once gives each cell its nearest taxi.
    R, C = len(grid), len(grid[0])
    dist = [[-1] * C for _ in range(R)]
    queue = deque()
    for r in range(R):
        for c in range(C):
            if grid[r][c] == "T":
                dist[r][c] = 0
                queue.append((r, c))
    spread(grid, dist, queue)
    return dist


# FOLLOW-UP 1: A new taxi appears.
# Answer: Start a BFS from the new taxi but only continue
# into cells where you improve the stored distance. The
# distances can only decrease, so the work is bounded by
# the cells whose answer actually changes.
def add_taxi(grid, dist, r, c):
    # a new taxi: only cells it beats are touched
    dist[r][c] = 0
    spread(grid, dist, deque([(r, c)]), better=True)


# FOLLOW-UP 3: Return which taxi is nearest, not only the
# distance.
# Answer: Store a source id with each cell in the BFS: a
# cell copies the id of the cell that first reached it.
# Ties go to whichever reached it first, so say how you
# want ties broken.
def nearest_taxi_ids(grid):
    # which taxi is nearest? each cell copies the id of
    # the
    # cell that reached it first (ties: the first to
    # arrive)
    R, C = len(grid), len(grid[0])
    owner = [[None] * C for _ in range(R)]
    queue, taxi_id = deque(), 0
    for r in range(R):
```

```python
        for c in range(C):
            if grid[r][c] == "T":
                owner[r][c] = taxi_id
                taxi_id += 1
                queue.append((r, c))
    while queue:
        r, c = queue.popleft()
        for dr, dc in STEPS:
            a, b = r + dr, c + dc
            inside = 0 <= a < R and 0 <= b < C
            if not inside or grid[a][b] == "#":
                continue
            if owner[a][b] is None:
                owner[a][b] = owner[r][c]
                queue.append((a, b))
    return owner


if __name__ == "__main__":
    grid = ["T...", ".#..", "...T"]
    d = taxi_distances(grid)
    # the second taxi at the bottom right is closer to (0,
```

```
    # 3)
    assert d[0] == [0, 1, 2, 2]
    assert d[2] == [2, 2, 1, 0]
    assert d[1][1] == -1
    # a new taxi in the middle of the top row
    add_taxi(grid, d, 0, 2)
    assert d[0] == [0, 1, 0, 1]
    assert d[2][0] == 2
    # unreachable cells stay -1
    walled = ["T#.", "##."]
    assert taxi_distances(walled)[0][2] == -1
    ids = nearest_taxi_ids(grid)
    assert ids[0][0] == 0 and ids[2][3] == 1
    assert ids[1][1] is None
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> list[list[int]]:
```

Sample call

```
solution([[0,0,0],[0,1,0],[0,0,0]])
```

Test cases

Input	Expected
`([[0,0,0],[0,1,0],[0,0,0]],)`	`[[2,1,2],[1,0,1],[2,1,2]]`
`([[1,0,0,0],[0,0,0,1],[0,0,0,0]],)`	`[[0,1,2,1],[1,2,1,0],[2,3,2,1]]`

STUDY PATH: question 92 of 167 (more practice)

Priority score 2.35 | Graph traversal: BFS and DFS | asked 271 days ago

MEDIUM | PracHub | Technical Screen | 2 of 3

Solve matrix groups and recipe inventory

POSTED 2026-01-22 | 271 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 33.8 | 30 DAYS 0 | PRIORITY 113/228

ABOUT THIS QUESTION

Topic: More practice - Graph traversal: BFS and DFS

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #2115 Find All Possible Recipes from Given Supplies

LeetCode match: No direct match; related LC 2115 Find All Possible Recipes from Given Supplies (inferred from overview)

Posted on PracHub: 2026-01-22 (updated 2026-05-12)

Original: https://prachub.com/coding-questions/solve-matrix-groups-and-recipe-inventory

QUESTION

Overview: This pair of problems evaluates block-wise grid processing and aggregation for minimum-value selection alongside dependency-resolution and resource-allocation reasoning for recipe inventory planning, testing competencies in array manipulation, tie-breaking and constraint handling, graph-based dependency management and cycle detection, and algorithmic complexity analysis; they are commonly asked to gauge an applicant's ability to implement correct, efficient solutions under input-size and edge-case constraints. They belong to the Coding & Algorithms domain with subdomains in data structures (arrays/grids), algorithm design and analysis, and graph theory, and primarily require practical application of algorithmic techniques with attention to time/space complexity rather than purely theoretical proofs.

APPROACH

Part 1: iterate block by block and take the min per group. Part 2: topological sort/DFS over recipe dependencies with an inventory map.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two parts. Matrix groups: how the grid is split into blocks (group size) and what is taken per block (the minimum). Recipes: can a recipe be made from supplies, do recipes need other recipes, are amounts counted or only availability (LeetCode 2115 is availability), how are cycles treated.

BRUTE FORCE TO SAY OUT LOUD

Matrix: scan each block directly. Recipes: recursive search from each recipe without memory.

OPTIMAL IDEA

Matrix: iterate block by block with nested loops and take the minimum of each block, O(mn). Recipes: build a graph from ingredients to recipes with in-degrees and run Kahn's algorithm starting from the supplies; a recipe becomes available when all its ingredients are; recipes on a cycle never become available. If quantities matter, DFS with memoization over recipes tracking inventory, and decide how to share a scarce ingredient between recipes.

DRY-RUN THESE CASES

A grid not divisible by the block size, a recipe whose ingredient is another recipe, a cycle between two recipes, a missing ingredient, an empty supply list.

LIKELY FOLLOW-UPS

Amounts per ingredient (resource allocation), the maximum number of each recipe, or the cheapest set of recipes.

PITFALLS

Treating availability as quantities. Not handling a cycle.

HOW TO THINK ABOUT IT

Blocks: visit each cell once and update the minimum of its block, indexed by (row // k, col // k). Recipes: this is a dependency graph; a recipe is makeable when all its ingredients are supplies or makeable recipes. Count unmet ingredients per recipe and process recipes as they become available (Kahn's algorithm); recipes on a cycle never become available.

Basis: Two parts (my copy has only the overview, so the rules are assumed): take the minimum of every k x k block of a matrix, and decide which recipes can be made from supplies where recipes may need other recipes (LeetCode 2115, Find All Possible Recipes).

BRUTE FORCE · Blocks: collect each block's values and call min

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid, h, w):
    answers = []
    for block in blocks(grid, h, w):
        cells = []
        for r, c in block:
            cells.append(grid[r][c])
        answers.append(min(cells))
    return answers
```

TIME	SPACE
O(m * n)	O(m * n)

WHY IT IS SLOW

Copies every block into a list. One pass that updates the minimum of cell (r // k, c // k) needs no copies.

BRUTE FORCE · Recipes: keep re-scanning until no recipe changes

BRUTE FORCE · PSEUDO-CODE

```
available = set(supplies)
while True:
    added = False
    for recipe in recipes:
        if recipe.name in available:
            continue
        needs = recipe.needs
        if all(i in available for i in needs):
            available.add(recipe.name)
            added = True
    if not added:
        break
```

TIME	SPACE
O(R^2 * I)	O(R)

WHY IT IS SLOW

Many passes over all recipes, O(R^2 * I). Counting each recipe's missing ingredients and processing items as they become available (Kahn's algorithm) is O(R + I).

OPTIMAL SOLUTION · Blocks: one pass updating a minimum per block

TIME	SPACE
O(m * n)	O(blocks)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Recipes: Kahn's algorithm on the ingredient graph

TIME	SPACE
O(R + I)	O(R + I)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Ingredients have quantities and a shared ingredient can only be used once.

Then availability is not enough: you also have to decide the order in which to spend scarce items. A priority order with a simulation (as coded) is a heuristic; the exact maximum is a resource-allocation problem (flow or integer programming).

FOLLOW-UP 2 Blocks that do not tile the matrix exactly.

The last block row or column is smaller; the single-pass update handles that automatically because the block index is r // k and c // k.

FOLLOW-UP 3 Detect which recipes are impossible because of a cycle.

After Kahn's algorithm, recipes whose counts never reached zero are either missing an ingredient or on a cycle; a DFS with colours over just those recipes tells the two cases apart.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve matrix groups and recipe inventory
#
# This pair of problems evaluates block-wise grid
# processing and aggregation for minimum-value selection
# alongside dependency-resolution and resource-allocation
# reasoning for recipe inventory planning, testing
# competencies in array manipulation, tie-breaking and
# constraint handling, graph-based dependency management
# and cycle detection, and algorithmic complexity
# analysis; they are commonly asked to gauge an
# applicant's ability to implement correct, efficient
# solutions under input-size and edge-case constraints.
# They belong to the Coding & Algorithms domain with
# subdomains in data structures (arrays/grids), algorithm
# design and analysis, and graph theory, and primarily
# require practical application of algorithmic techniques
# with attention to time/space complexity rather than
# purely theoretical proofs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Ingredients have quantities and a shared ingredient
# can only be used once.
# 2. Blocks that do not tile the matrix exactly.
# 3. Detect which recipes are impossible because of a
# cycle.
#
# INTERVIEW NOTES
# Blocks: visit each cell once and update the minimum of
# its block, indexed by (row // k, col // k). Recipes:
# this is a dependency graph; a recipe is makeable when
# all its ingredients are supplies or makeable recipes.
# Count unmet ingredients per recipe and process recipes
# as they become available (Kahn's algorithm); recipes on
# a cycle never become available.
#
# BRUTE FORCE (say it first): Blocks: collect each block's
# values and call min
# For every block gather its cells into a list and take
# the minimum. Correct and the oracle; it copies every
# block. Time O(m * n). Space O(m * n).


from collections import defaultdict, deque
```

```
# BRUTE FORCE (say it first): Blocks: collect each block's
# values and call min | time O(m * n)
def block_mins_collect(matrix, k):
    m, n = len(matrix), len(matrix[0])
    return [
        [
            min(
                matrix[r][c]
                for r in range(i, min(i + k, m))
                for c in range(j, min(j + k, n))
            )
            for j in range(0, n, k)
```

```
        ]
        for i in range(0, m, k)
    ]


# BASE QUESTION (optimal): Blocks: one pass updating a
# minimum per block
# Idea: Each cell updates the minimum of block (r // k, c
# // k). No block list is built; the last blocks may be
# smaller when k does not divide the size.
def block_mins(matrix, k):
    # the minimum of every k x k block; edge blocks may be
    # smaller, which r // k and c // k handle by
    # themselves
    m, n = len(matrix), len(matrix[0])
    rows, cols = (m + k - 1) // k, (n + k - 1) // k
    out = [[float("inf")] * cols for _ in range(rows)]
    for r in range(m):
        for c in range(n):
            cell = out[r // k][c // k]
            out[r // k][c // k] = min(cell, matrix[r][c])
    return out


# PART 2: which recipes can be made
def possible_recipes(recipes, supplies):
    # recipes: name -> list of ingredients (which may be
    # other
    # recipes). Kahn's algorithm over missing-ingredient
    # counts.
    have = set(supplies)
    missing, users = {}, defaultdict(list)
    for name, items in recipes.items():
```

```
        missing[name] = 0
        for item in set(items):
            if item not in have:
                missing[name] += 1
                users[item].append(name)
    queue = deque(r for r, m in missing.items() if m == 0)
    made = []
    while queue:
        name = queue.popleft()
        made.append(name)
        # a finished recipe is a new ingredient for its
        # users
        for other in users[name]:
            missing[other] -= 1
            if missing[other] == 0:
                queue.append(other)
    return made


# FOLLOW-UP 1: Ingredients have quantities and a shared
# ingredient can only be used once.
# Answer: Then availability is not enough: you also have
# to decide the order in which to spend scarce items.
def recipes_with_quantities(recipes, needs, stock):
    # ingredients have amounts and stock is used up. Make
    # each recipe at most once, in priority order, while
    # the
    # stock allows (a made recipe is new stock). This is a
    # heuristic: the exact best choice is a flow problem.
    stock, made = dict(stock), []
    progress = True
    while progress:
        progress = False
        for name, need in zip(recipes, needs):
            if name in made:
                continue
            enough = all(stock.get(x, 0) >= q
                         for x, q in need.items())
            if enough:
```

```
                for x, q in need.items():
                    stock[x] -= q
                stock[name] = stock.get(name, 0) + 1
                made.append(name)
                progress = True
    return made
```

```


# FOLLOW-UP 3: Detect which recipes are impossible because
# of a cycle.
# Answer: After Kahn's algorithm, recipes whose counts
# never reached zero are either missing an ingredient or
# on a cycle; a DFS with colours over just those recipes
# tells the two cases apart.
def why_impossible(recipes, supplies):
    # for each recipe that cannot be made: a missing raw
    # ingredient, or a cycle
    made = set(possible_recipes(recipes, supplies))
    known = set(supplies) | set(recipes)
    reasons = {}
    for name, items in recipes.items():
        if name in made:
            continue
        gone = [i for i in items if i not in known]
        reasons[name] = "cycle"
        if gone:
            reasons[name] = "missing " + gone[0]
    return reasons


if __name__ == "__main__":
    matrix = [[5, 3, 8, 1], [2, 9, 4, 7], [6, 0, 3, 2]]
    # 2 x 2 blocks, the last row of blocks is only 1 row
    # tall
    assert block_mins(matrix, 2) == [[2, 1], [0, 2]]
    assert block_mins([[4]], 3) == [[4]]
    recipes = {"bread": ["flour", "yeast"],
               "sandwich": ["bread", "meat"],
               "burger": ["bread", "meat", "bun"]}
    stock = ["flour", "yeast", "meat"]
    got = possible_recipes(recipes, stock)
    assert got == ["bread", "sandwich"]
    # two recipes that need each other, and a missing item
    cyc = {"a": ["b"], "b": ["a"], "c": ["x"]}
    assert possible_recipes(cyc, []) == []
    assert why_impossible(cyc, []) == {"a": "cycle",
                                       "b": "cycle",
                                       "c": "missing x"}
    # amounts: one bread uses the only 2 flour, so the
    # cake
    # (which needs 2 flour too) cannot be made
```

```
    names = ["bread", "cake"]
    needs = [{"flour": 2}, {"flour": 2}]
    two = {"flour": 2}
    made = recipes_with_quantities(names, needs, two)
    assert made == ["bread"]
    four = {"flour": 4}
    both = recipes_with_quantities(names, needs, four)
    assert both == ["bread", "cake"]
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> list[int]:
```

Sample call

```
solution([[5,4,3,9,8,7],[6,2,8,5,4,6],[7,3,9,8,2,5],[10,11
,12,0,3,4],[9,8,7,6,5,4],[3,2,9,7,8,9]])
```

Test cases

Input	Expected
([[5,4,3,9,8,7],[6,2,8,5,4,6],[7,3,9, 8,2,5],[10,11,12,0,3,4],[9,8,7,6,5,4] ,[3,2,9,7,8,9]],)	(0, 3, 3)
([[-1,5,6,-1,7,8],[9,10,11,12,13,14], [15,16,17,18,19,20],[21,22,23,24,25,2 6],[27,28,29,30,31,32],[33,34,35,36,3 7,38]],)	(-1, 0, 0)

STUDY PATH: question 93 of 167 (more practice)

Priority score 2.16 | Graph traversal: BFS and DFS | asked 251 days ago

MEDIUM | IN APP LC 261 | PracHub | Technical Screen | 3 of 3

Validate parent array forms a tree

POSTED 2026-02-11 | 251 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 29 | 30 DAYS 0 | PRIORITY 128/228

ABOUT THIS QUESTION

Topic: More practice - Graph traversal: BFS and DFS

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #261 Graph Valid Tree

LeetCode match: LC 261 Graph Valid Tree

Posted on PracHub: 2026-02-11 (updated 2026-04-23)

Original:
https://prachub.com/coding-questions/validate-parent-array-forms-a-tree

QUESTION

Problem

You are given an integer array **`parent`** of length **`n`** describing a directed parent pointer for each node **`i`** (nodes are labeled **`0..n-1`**).

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these **`n`** nodes form a **valid rooted tree**. A valid rooted tree must satisfy all of the following: 1.

Exactly one root (exactly one index **`i`** with **`parent[i] = -1`**). 2.
No cycles (following parent pointers from any node must eventually reach the root). 3.
Connectivity (every node is reachable from the root; equivalently, every node has exactly one simple path to the root).

Input

- **`parent`**: integer array of length **`n`** where each value is either **`-1`** or in **`[0, n-1]`**.

Output

- Return **`true`** if **`parent`** represents a valid rooted tree; otherwise return **`false`**.

Notes

/ Edge Cases - **`n`** can be 1 (then the only valid tree is **`parent[0] = -1`**).

- Self-parenting like **`parent[i] = i`** is invalid.
- Multiple **`-1`** entries (multiple roots) is invalid.
- A cycle among non-root nodes is invalid.
- A disconnected component (some nodes not reachable from the root) is invalid.

You are given an integer array **`parent`** of length **`n`** describing a parent pointer for each node **`i`** where nodes are labeled **`0`** to **`n-1`**.

QUESTION

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these nodes form a valid rooted tree. A valid rooted tree must satisfy all of the following:

1. Exactly one root.
2. No cycles.
3. Every node belongs to the same connected structure and has a path to the root. Return **`True`** if **`parent`** represents a valid rooted tree; otherwise return **`False`**.

Constraints
1 <= n <= 2 * 10^5Each **`parent[i]`** is either **`-1`** or an integer in **`[0, n-1]parent[i] = i`** is invalid

EXAMPLES

```
Input: [-1, 0, 0, 1, 1]
Expected Output: True
```

Explanation: Node 0 is the only root. All other nodes are connected under it, and there are no cycles.

```
Input: [2, -1, 1, 2]
Expected Output: True
Explanation: Node 1 is the root. The paths are
    0->2->1, 2->1, and 3->2->1, so every node reaches
    the same root.
```

Hints

First count how many roots there are. A valid tree must have exactly one **`-1`** entry.Build children lists from the parent array, then traverse from the root. If you do not visit every node, the structure is disconnected or contains a cycle away from the root.

APPROACH

Exactly one root (parent = -1 or self), every other node has one parent; detect cycles by walking up with a visited set or union-find; all nodes must reach the root. O(n).

HOW TO THINK ABOUT IT

Exactly one -1 gives the root; then the rest is whether every node reaches it. Following parents from each node can loop (a cycle) or run out of range. Do it with memoised walks so every node is processed once: mark nodes on the current walk, and if the walk meets one of them again it is a cycle; if it meets a node already known to reach the root, all nodes on the walk reach it too.

Basis: LeetCode 261 (Graph Valid Tree) in parent-pointer form: exactly one root (-1), no cycles, every node reaches the root.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_tree(parent):
    n = len(parent)
    roots = [i for i in range(n)
             if parent[i] == -1]
    if len(roots) != 1:
        return False
    if any(p >= n for p in parent):
        return False
    for node in range(n):
        cur, steps = node, 0
        while cur != -1 and steps <= n:
            cur = parent[cur]
            steps += 1
        if cur != -1:
            return False  # a cycle
    return True
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Every node walks its whole chain, O(n^2). Marking nodes as 'on this walk' or 'verified' lets every node be processed once, O(n).

OPTIMAL SOLUTION · One root, then colour each node once while walking up

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The graph is given as undirected edges (LeetCode 261).

A tree has exactly n - 1 edges and no cycle: check the edge count, then union endpoints and fail on an edge that joins two nodes already connected. O(n alpha(n)).

FOLLOW-UP 2 Report the reason it is invalid.

Return which rule failed: number of roots, an out-of-range parent, or the node where the cycle was detected (the node that was met twice on one walk).

FOLLOW-UP 3 A binary tree given as child arrays (LeetCode 1361).

Check that exactly one node has in-degree 0, every other node has in-degree 1, and that a traversal from the root visits all n nodes.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Validate parent array forms a tree
#
# ## Problem You are given an integer array parent of
# length n describing a directed parent pointer for each
# node i (nodes are labeled 0..n-1). - parent[i] is the
# parent of node i. - Exactly one node should be the root,
# indicated by parent[root] = -1. Determine whether these
# n nodes form a **valid rooted tree**. A valid rooted
# tree must satisfy all of the following: 1. **Exactly one
# root** (exactly one index i with parent[i] = -1). 2.
# **No cycles** (following parent pointers from any node
# must eventually reach the root). 3. **Connectivity**
# (every node is reachable from the root; equivalently,
# every node has exactly one simple path to the root). ##
# Input - parent: integer array of length n where each
# value is either -1 or in [0, n-1]. ## Output - Return
# true if parent represents a valid rooted tree; otherwise
# return false.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The graph is given as undirected edges (LeetCode
# 261).
# 2. Report the reason it is invalid.
# 3. A binary tree given as child arrays (LeetCode 1361).
#
# INTERVIEW NOTES
# Exactly one -1 gives the root; then the rest is whether
# every node reaches it. Following parents from each node
# can loop (a cycle) or run out of range. Do it with
# memoised walks so every node is processed once: mark
# nodes on the current walk, and if the walk meets one of
# them again it is a cycle; if it meets a node already
# known to reach the root, all nodes on the walk reach it
# too.
#
# BRUTE FORCE (say it first): Follow the parent chain from
# every node, up to n steps
# From each node follow parents for at most n steps; if
# you do not reach the root in n steps there is a cycle.
# Correct and the oracle; each node walks its whole chain
# separately. Time O(n^2). Space O(1).


from collections import defaultdict
```

```
# BRUTE FORCE (say it first): Follow the parent chain from
# every node, up to n steps | time O(n^2)
def valid_tree_walk(parent):
    n = len(parent)
    roots = [i for i, p in enumerate(parent) if p == -1]
    if len(roots) != 1 or any(
        p != -1 and not (0 <= p < n) for p in parent
    ):
        return False
    for i in range(n):
        x, steps = i, 0
        while parent[x] != -1 and steps <= n:
            x = parent[x]
            steps += 1
        if parent[x] != -1:
            return False
    return True


# HELPERS for the base question
```

```
# FOLLOW-UP 2: Report the reason it is invalid.
# Answer: Return which rule failed: number of roots, an
# out-of-range parent, or the node where the cycle was
# detected (the node that was met twice on one walk).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def reason(parent):
    # None if valid, else the first rule that fails
    n = len(parent)
    roots = parent.count(-1)
    if roots != 1:
        return f"{roots} roots (need exactly 1)"
    for i, p in enumerate(parent):
        if p != -1 and not 0 <= p < n:
            return f"node {i}: out-of-range parent {p}"
        if p == i:
            return f"node {i} is its own parent"
    # 0 = unseen, 1 = on the current walk, 2 = checked
    state = [0] * n
    for start in range(n):
        path, x = [], start
        while x != -1 and state[x] == 0:
            state[x] = 1
            path.append(x)
            x = parent[x]
        # meeting a node twice on one walk is a cycle
        if x != -1 and state[x] == 1:
            return f"cycle through node {x}"
        for v in path:
            state[v] = 2
    return None


# BASE QUESTION (optimal): One root, then colour each node
# once while walking up
# Idea: Check there is exactly one -1 and all other values
# are in range. Then walk up from each unvisited node
# marking nodes as 'on this walk'.
def validate_parent_array(parent):
    return reason(parent) is None


# FOLLOW-UP 1: The graph is given as undirected edges
# (LeetCode 261).
# Answer: A tree has exactly n - 1 edges and no cycle:
# check the edge count, then union endpoints and fail on
# an edge that joins two nodes already connected. O(n
# alpha(n)).
def valid_tree_edges(n, edges):
    # LeetCode 261: undirected edges
    if len(edges) != n - 1:
        return False
    root = list(range(n))

    def find(x):
        while root[x] != x:
            root[x] = root[root[x]]
            x = root[x]
        return x

    for u, v in edges:
        a, b = find(u), find(v)
        # an edge inside a group closes a cycle
        if a == b:
            return False
        root[a] = b
    return True


# FOLLOW-UP 3: A binary tree given as child arrays
# (LeetCode 1361).
# Answer: Check that exactly one node has in-degree 0,
# every other node has in-degree 1, and that a traversal
# from the root visits all n nodes.
def valid_binary_tree(n, left, right):
    # LeetCode 1361: child arrays, -1 means no child
    indeg = [0] * n
    for kids in (left, right):
        for c in kids:
            if c != -1:
                indeg[c] += 1
    roots = [i for i in range(n) if indeg[i] == 0]
    if len(roots) != 1 or max(indeg) > 1:
        return False
    seen, stack = {roots[0]}, [roots[0]]
    while stack:
        u = stack.pop()
        for c in (left[u], right[u]):
            if c != -1:
                if c in seen:
                    return False
                seen.add(c)
                stack.append(c)
    # every node must be reachable from the root
    return len(seen) == n


if __name__ == "__main__":
    assert validate_parent_array([-1, 0, 0, 1])
    assert validate_parent_array([-1])
    # two roots, none, and a cycle 1 -> 2 -> 1
    assert not validate_parent_array([-1, -1])
    assert not validate_parent_array([1, 0])
    assert "cycle" in reason([-1, 2, 1])
    assert not validate_parent_array([-1, 5])
    assert "out-of-range" in reason([-1, 5])
    assert "own parent" in reason([-1, 1])
    star = [(0, 1), (0, 2), (0, 3), (1, 4)]
    assert valid_tree_edges(5, star)
    loop = [(0, 1), (1, 2), (2, 0)]
    assert not valid_tree_edges(4, loop)
    left, right = [1, -1, 3, -1], [2, -1, -1, -1]
    assert valid_binary_tree(4, left, right)
    assert not valid_binary_tree(2, [1, 0], [-1, -1])
```

FUNCTION AND TEST CASES

Signature

```
def solution(parent: list[int]) -> bool:
SAMPLE INPUT
[-1, 0, 0, 1, 1]
```

Test cases

Input	Expected
[-1, 0, 0, 1, 1]	True
[2, -1, 1, 2]	True